

Machine Learning Based Integrated Solution for Smart Pepper Farming in Sri Lanka

Project ID: 25-26J-479

Checklist 2 - Deployment Report

BSc (Hons) in Information Technology Specializing in
Software Engineering

Department of Software Engineering
Sri Lanka Institute of Information Technology

Table of Contents

Executive Summary & Objectives.....	3
Purpose.....	3
Outcomes	4
Key Metrics.....	4
Goals and Scope.....	4
Deployment Process & Activities	5
Pre-Deployment Setup	5
Containerization	5
Step-by-step Deployment Procedure	6
Frontend Deployment (Website).....	6
Backend Deployment.....	6
Machine Learning Models Deployment	6
Testing Activities	7
Unit Testing.....	7
Integration Testing	7
User Acceptance Testing (UAT)	7
Backup and recovery plans	7
Deployment Checklist.....	8
Pre-deployment checklist.....	8
During deployment checklist	8
Post-deployment checklist	8
Issues and Resolutions	9
Issues encountered	9
Solutions applied.....	9
Impact on deployment.....	9
Performance and Monitoring	10
Performance metrics	10
Monitoring tools used	10
Monitoring results	10

Post-Deployment Activities	11
Validation and testing.....	11
Documentation updates.....	11
Training & Support	11
Lessons Learned.....	12
Successes.....	12
Challenges faced	12
Areas for Improvements	12
Appendices.....	14
Appendix A - Deployment Architecture Diagram	14
Appendix B - Google Cloud Logs	15
Appendix C – Vercel Deployment and Performance Monitoring.....	16

Table of Figures

Figure 1: Deployment Architecture Diagram	14
Figure 2: System Monitoring Dashboard with Performance Metrics	15
Figure 3: Application Metrics and System Activity Dashboard	15
Figure 4: Deployed Docker Containers in Cloud Environment.....	16
Figure 5: Vercel Deployment Overview	16
Figure 6: Vercel Speed Insights Performance Metrics.....	17

Executive Summary & Objectives

This project presents a Machine Learning-Based Integrated Solution for Smart Pepper Farming in Sri Lanka, developed to address key agricultural challenges through data-driven technologies. The system incorporates multiple intelligent components, including:

- Yield Prediction
- Pepper Quality Grading
- Price Prediction and Blockchain Integrity
- Disease Detection

The system integrates agricultural data, environmental factors, image-based analysis, and historical market trends to provide accurate predictions and decision support for pepper farmers. The platform is designed as a unified ecosystem where all components interact through a centralized backend.

Farmers are able to upload images to obtain predictions for yield, disease identification, and quality grading. Both farmers and exporters can access local market prices and export price predictions based on the input features provided to the system. The solution is deployed using Google Cloud Platform to host machine learning models and backend services, while the administrative web interface is deployed on Vercel to enable system monitoring and management.

Purpose

The purpose of deployment is to:

- Make the system accessible to farmers and exporters via a mobile application
- Provide an admin web interface for system monitoring and management
- Host ML models and backend services on a scalable cloud infrastructure
- Enable real-time predictions for yield, disease detection, grading and price forecasting
- Support data-driven decision-making in pepper farming

Outcomes

- Farmers can upload images and receive predictions for yield, disease detection, and quality grading
- Farmers and exporters can access predicted local and export market prices based on input data
- Admin users can manage and monitor the overall system through the web interface
- Backend services provide real-time inference from all ML models
- The system supports improved decision-making in farming and trading activities

Key Metrics

- Prediction accuracy of yield, quality grading, price and disease detection models
- API response time and image processing latency
- System uptime and availability
- Number of prediction requests (image uploads and price queries)
- Admin system monitoring efficiency

Goals and Scope

Included:

- Farmers are provided with image upload functionality to obtain predictions for yield, disease detection and quality grading
- Price prediction for both local and export markets
- Mobile application usage for farmers and exporters
- Admin web interface deployed on Vercel is provided for system management
- Deployment of ML models and backend on Google Cloud Platform
- Real-time prediction and data processing

Excluded:

- Multi-region cloud deployment
- Advanced auto-scaling mechanisms

Deployment Process & Activities

Pre-Deployment Setup

- **Model Development and Validation**

Machine learning models were developed using Python and validated to ensure accurate predictions for yield, disease detection, quality grading, and price forecasting.

- **ML API Development using FastAPI**

Each trained model was integrated with FastAPI to expose prediction functionalities through RESTful APIs.

- **Backend Development using Node.js**

A separate backend was developed using Node.js to handle application logic, user requests, and communication between frontend and ML services.

- **Frontend Preparation**

The admin web interface was finalized and configured for deployment.

- **Cloud Environment Setup**

Google Cloud Platform resources were configured to host backend services and ML model APIs.

- **Environment Configuration**

Environment variables, API endpoints and security settings were defined for secure system operation.

Containerization

- Docker was used to containerize both the Node.js backend and the Python-based FastAPI machine learning services.
- Each service, including individual machine learning models and the backend, was packaged into separate containers to ensure modularity.

- All dependencies required for Node.js and Python environments were included within Docker images to maintain consistency.
- Containerization ensured that the system could run reliably across different environments, including local development and cloud deployment.

Step-by-step Deployment Procedure

Frontend Deployment (Website)

- The admin web interface built using React was deployed on Vercel for fast and reliable hosting.
- Continuous deployment was enabled by connecting the project repository to the Vercel platform.
- The deployed web interface allows administrators to monitor system performance and manage operations.
- Vercel provides optimized performance, fast loading times, and automatic updates for the frontend application.

Backend Deployment

- The backend developed using Node.js was deployed on Google Cloud Platform to ensure scalability and reliability.
- The backend handles requests from both the mobile application and the admin web interface.
- Secure API endpoints were implemented to ensure safe data transmission.
- The cloud environment enables the backend to efficiently manage multiple concurrent requests.

Machine Learning Models Deployment

- Machine learning models were developed using Python and deployed as independent services.
- Each model was integrated with FastAPI to provide real-time prediction APIs.
- All FastAPI services were deployed on Google Cloud Platform.
- The models process both image inputs and structured data to generate predictions.

Testing Activities

Unit Testing

- Individual components such as Node.js APIs and FastAPI endpoints were tested independently.
- Each function and module was validated to ensure correct outputs for given inputs.

Integration Testing

- The interaction between frontend, backend, and machine learning services was thoroughly tested.
- End-to-end data flow was validated to ensure accurate communication across all components.

User Acceptance Testing (UAT)

- The system was tested by real users including farmers, exporters, and administrators.
- Improvements were made based on user feedback to enhance system effectiveness.

Backup and recovery plans

- Version control is maintained using Git to track changes and manage different versions of the system.
- Previous Docker images are stored to ensure that stable versions of the application can be reused when needed.
- Regular backups of application data and configurations are maintained within the cloud environment to prevent data loss.
- Rollback is performed by redeploying previously stable versions of the backend and machine learning services.
- Cloud infrastructure is utilized to ensure high availability and minimize system downtime.

Deployment Checklist

Pre-deployment checklist

- All machine learning models are trained, tested, and validated for accuracy.
- FastAPI endpoints and the Node.js backend are fully implemented and tested locally.
- Docker containers for backend and ML services are built and verified.
- Google Cloud Platform environment and configurations are properly set up.
- Frontend applications are completed and tested for functionality.

During deployment checklist

- Backend and FastAPI ML services are successfully deployed on Google Cloud Platform.
- The React admin web application is deployed on Vercel without errors.
- API connections between frontend, backend, and ML services are functioning correctly.
- System logs and performance are monitored during deployment.

Post-deployment checklist

- The system is tested in the production environment to ensure full functionality.
- All APIs and prediction services are verified for accuracy and response time.
- Frontend applications are tested for usability and responsiveness.
- Backup and rollback mechanisms are verified.
- User feedback is collected and necessary improvements are applied.

Issues and Resolutions

Issues encountered

- Deployment errors were encountered due to dependency mismatches in Docker containers.
- A port conflict issue occurred during backend deployment, preventing the service from running properly.
- Handling image uploads and processing caused latency issues in some scenarios.
- Configuration issues related to environment variables and API endpoints affected initial deployment.

Solutions applied

- Docker configurations were corrected by aligning dependencies and rebuilding stable images.
- The backend port conflict issue was resolved by changing the port number to an available port.
- Image processing was optimized to reduce latency and improve response time.
- Environment variables and configurations were carefully verified and updated.

Impact on deployment

- The system achieved stable communication between frontend, backend and machine learning services.
- Deployment reliability improved after resolving configuration and dependency issues.
- The backend service was successfully deployed after fixing the port conflict issue.

Performance and Monitoring

Performance metrics

The performance of the system was evaluated using several key metrics to ensure reliability and effectiveness. The accuracy of the machine learning models was a primary factor, particularly for yield prediction, disease detection, quality grading and price prediction. In addition, API response time was measured to ensure efficient communication between the frontend, backend and machine learning services. Special attention was given to image processing latency, as image-based predictions require more computational resources. System uptime and availability were also monitored to ensure that users could access the platform without interruptions. Furthermore, the number of user requests and prediction calls was analyzed to assess the system's ability to handle real-world usage.

Monitoring tools used

A range of monitoring tools was utilized to track system performance and ensure smooth operation after deployment. Google Cloud Platform's built-in monitoring and logging services were used to observe backend performance, API usage, and machine learning service behavior. These tools provided insights into system health, errors, and resource utilization. Vercel analytics were used to monitor the performance and deployment status of the admin web interface. Additionally, console logging and debugging tools were used during development and deployment to identify and resolve issues effectively.

Monitoring results

The monitoring process indicated that the system performed reliably under normal operating conditions. API response times remained stable, allowing efficient communication between system components. The machine learning models delivered accurate predictions with acceptable processing times. Image-based predictions showed slightly higher latency compared to other operations, but the performance remained within acceptable limits. The system maintained high availability due to the robustness of the cloud infrastructure. Any minor issues identified during monitoring were promptly addressed, resulting in improved overall system stability and performance.

Post-Deployment Activities

Validation and testing

Validation and testing were carried out after deployment to ensure that the system functions correctly and meets user requirements. Different testing methods were used to verify the performance, accuracy, and integration of all system components.

- The system was tested in the production environment to ensure all features function correctly.
- Machine learning models were validated using real input data to confirm prediction accuracy.
- API endpoints were tested to ensure proper communication between frontend, backend, and ML services.
- Image upload and processing functionalities were verified for correctness and performance.

Documentation updates

Documentation was updated to reflect the final deployed system, ensuring that all components, configurations, and processes are clearly recorded for future reference and maintenance.

- System architecture and deployment details were documented clearly.
- API documentation was updated to describe endpoints and usage.
- Configuration settings and environment variables were recorded.

Training & Support

Training and support were provided to ensure that users can effectively use the system and understand its functionalities. Guidance was given to both end users and administrators.

- Farmers were guided on how to use the mobile application and upload inputs.
- Team members were trained on the deployment process to ensure proper system handling and maintenance.
- Support was offered to resolve user issues and technical difficulties.

Lessons Learned

Successes

The deployment of the system was successfully completed, achieving the intended objectives and ensuring smooth operation across all components. The integration of different technologies and services resulted in a functional and reliable platform.

- All machine learning models were successfully deployed and provided accurate predictions in real time.
- The integration between the Flutter mobile application, Node.js backend and FastAPI services functioned effectively.
- The admin web interface enabled efficient system monitoring and management.

Challenges faced

Several challenges were encountered during the deployment process, mainly related to system integration, configuration, and performance. These challenges required careful troubleshooting and adjustments.

- A port conflict occurred during backend deployment, which prevented the service from running properly.
- Managing dependencies in Docker containers required additional configuration efforts.
- Image processing tasks resulted in higher latency compared to other operations.
- Configuration of environment variables and API endpoints caused initial deployment difficulties.

Areas for Improvements

Although the system was successfully deployed, there are several areas that can be improved to enhance performance, scalability and user experience in future developments.

- Image processing speed can be further optimized to reduce latency.
- Advanced scaling mechanisms can be implemented to handle higher user loads.
- Additional security features can be introduced to enhance data protection.

- A CI/CD pipeline can be implemented to automate the build, testing and deployment process.
- Automated testing can be added to ensure system reliability and reduce manual testing efforts.

Appendices

Appendix A - Deployment Architecture Diagram

This diagram illustrates the overall deployment architecture of the system. It shows how the mobile application and admin web interface interact with the Node.js backend and independently deployed FastAPI-based machine learning services hosted on Google Cloud Platform. The diagram also highlights the use of Docker containers for service deployment and direct communication between client applications and ML services for real-time predictions.

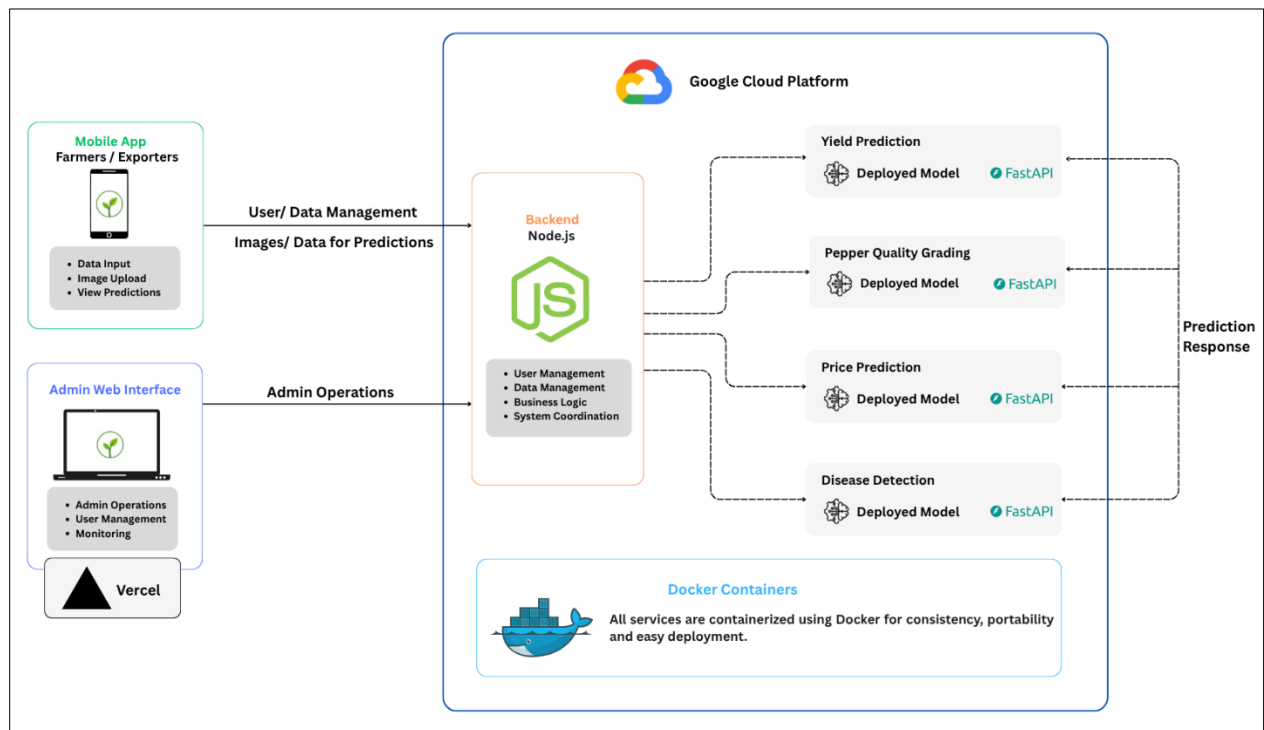


Figure 1: Deployment Architecture Diagram

Appendix B - Google Cloud Logs

The below figures present system logs generated by the backend and machine learning services deployed on Google Cloud Platform, showing request handling and overall system activity for monitoring and debugging purposes.

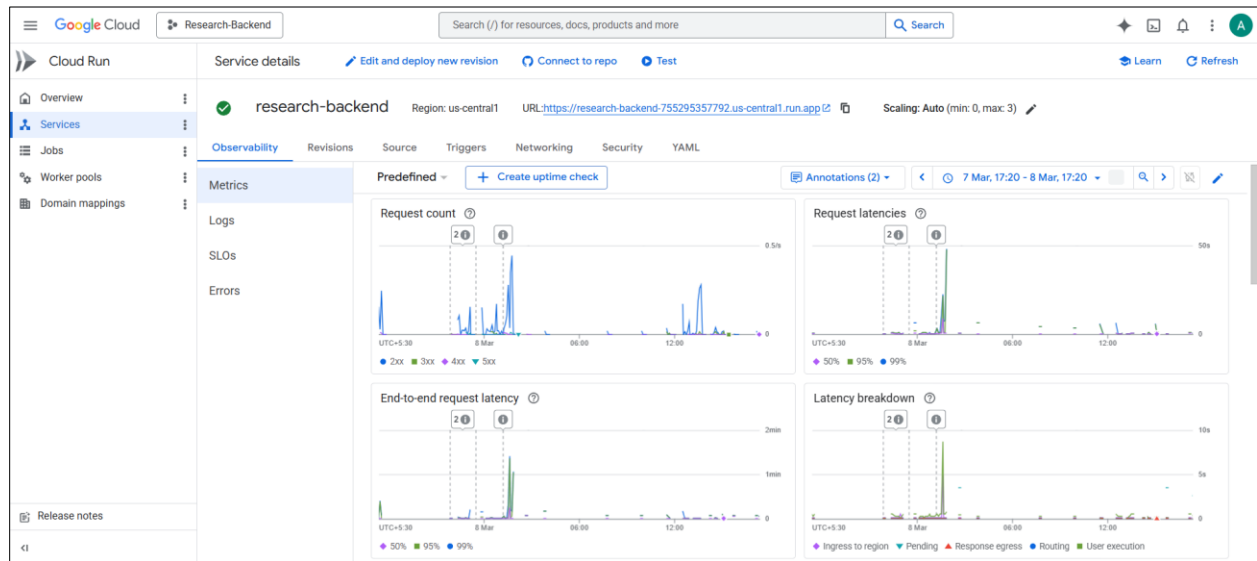


Figure 2: System Monitoring Dashboard with Performance Metrics

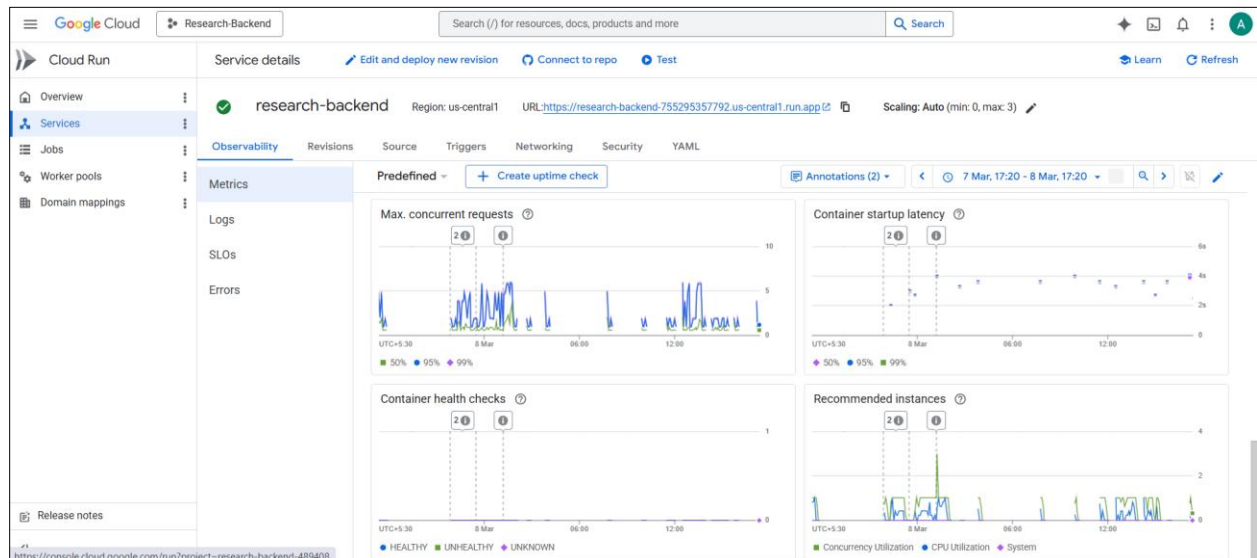


Figure 3: Application Metrics and System Activity Dashboard

The screenshot shows the Google Cloud Run 'Services' page. A sidebar on the left contains navigation links: Overview, Services (selected), Jobs, Worker pools, and Domain mappings. The main content area has a header with 'Services', 'Deploy container', 'Connect repo', and 'Write a function' buttons. Below this is a table of services. The table has columns for Name, Deployment type, Req/sec, Region, Cost, Authentication, Ingress, Last deployed, and Deployed by. Five services are listed, all with a deployment type of '(A) Container' and 'Public access' authentication. The services are: disease-detection-optimized, price-prediction, quality-grading, research-backend, and yield-prediction. The last two were deployed on 10 Mar 2026, while the others were on 9 Mar 2026.

Name	Deployment type	Req/sec	Region	Cost	Authentication	Ingress	Last deployed	Deployed by
disease-detection-optimized	(A) Container	0	eu-west1	—	Public access	All	9 Mar 2026	ashikarid2002@g
price-prediction	(A) Container	0	eu-west1	—	Public access	All	9 Mar 2026	ashikarid2002@g
quality-grading	(A) Container	0	eu-west1	—	Public access	All	9 Mar 2026	ashikarid2002@g
research-backend	(A) Container	0	us-central1	—	Public access	All	9 Mar 2026	ashikarid2002@g
yield-prediction	(A) Container	0	eu-west1	—	Public access	All	10 Mar 2026	madameegama

Figure 4: Deployed Docker Containers in Cloud Environment

Appendix C – Vercel Deployment and Performance Monitoring

This presents the deployment and performance monitoring details of the admin web interface hosted on Vercel, including system overview and performance insights used to evaluate application efficiency.

The screenshot shows the Vercel 'Overview' page for a project named 'ceylon-pepper'. The left sidebar contains a navigation menu with links: Overview, Deployments, Logs, Analytics, Speed Insights, Observability, Firewall, CDN, Domains, Integrations, Storage, Flags, Agent, and AI Gateway. The main content area is titled 'Production Deployment' and shows a preview of the 'CeylonPepper' application. To the right of the preview, deployment details are shown: 'ceylon-pepper-4cf8rwt2d-janudis-projects-e08d75ab.vercel.app', 'Ready' status, and deployment by 'JanudiAdhikari' on 'Apr 13'. Below this is a 'Production Checklist' with items like 'Connect Git Repository', 'Add Custom Domain', 'Preview Deployment', 'Enable Web Analytics', and 'Enable Speed Insights'. To the right of the checklist are 'Observability' and 'Analytics' sections. The 'Observability' section shows 'Edge Requests' at 0, 'Function Invocations' at 0, and 'Error Rate' at 0%. The 'Analytics' section has a toggle to 'Track visitors and page views' which is currently disabled.

Figure 5: Vercel Deployment Overview

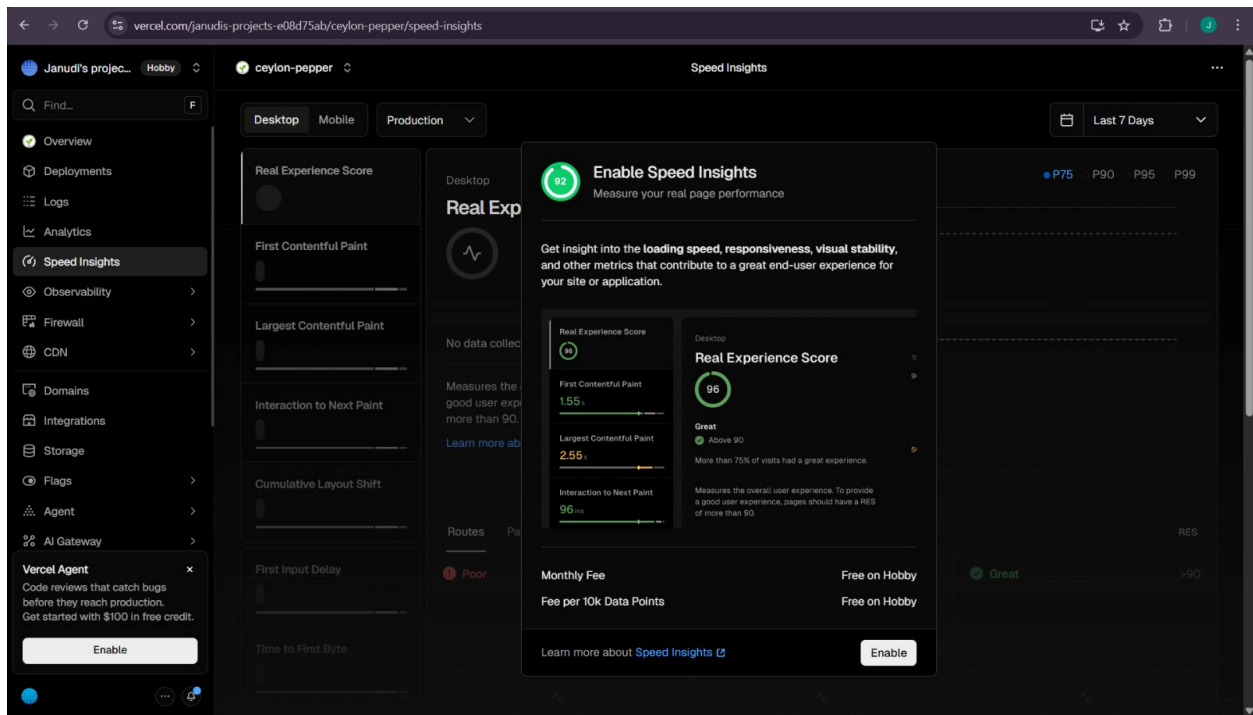


Figure 6: Vercel Speed Insights Performance Metrics